HIGH PERFORMANCE COMPUTING 101

HPC Computing @ IU (Module 2)

BRING ON TOMORROW

Module 2 — Lesson 2

Using your accounts

BRING ON TOMORROW

Overview

- Accessing Research Desktop
- Accessing HPC with SSH
- Finding and Loading Software
- Working with Batch Jobs
- Managing a SLURM Job

Accessing Research Desktop

- Accessing Research Desktop
- Accessing HPC with SSH
- Finding and Loading Software
- Working with Batch Jobs
- Managing a SLURM Job

Accessing Research Desktop

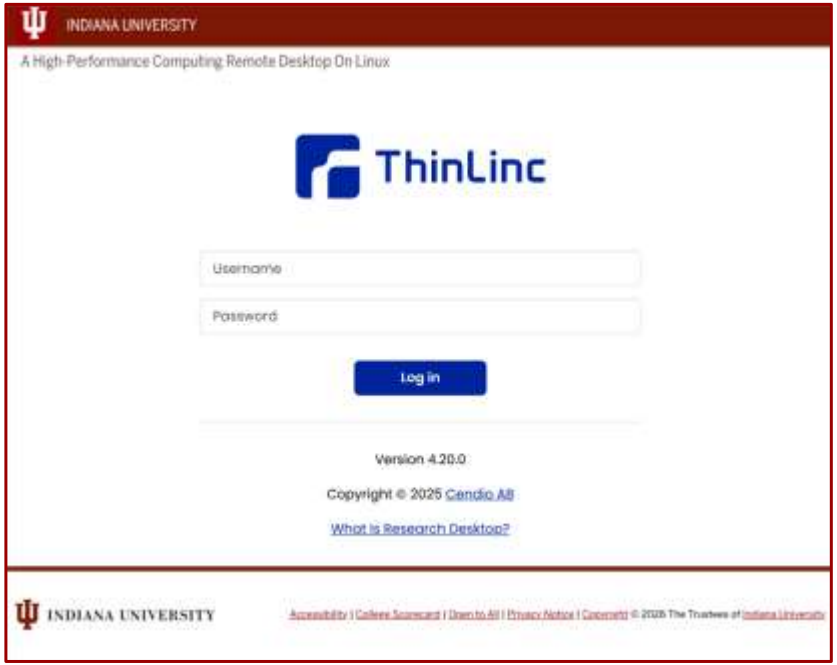
- RED is available via both
  - a web interface
  - full-featured desktop client (called ThinLinc) available
    - Windows, MacOS & Linux are supported
  - download from https://www.cendio.com/thinlinc/download/

- We will start with the web interface
  - you'll only need a browser

- Open your browser and go to https://red.uits.iu.edu/

Accessing Research Desktop

- Expect to see a login window like this
- Use your IU Account username and password
- Proceed through the Duo 2FA process to log in



Accessing Research Desktop

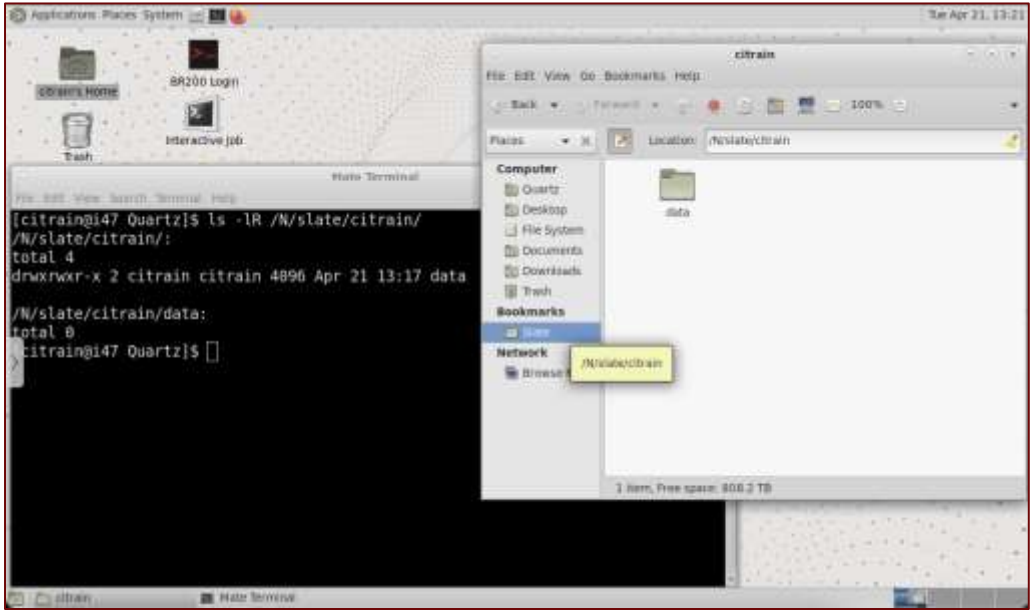
- The desktop will appear like this
- Icons on the desktop and drop-down menus on the upper left
- Notice the files app (called Caja) upper left

Accessing Research Desktop

- Allows you to manage files as you would in Windows or MacOS
- Of course, command line access is available, too
- Note the folder icon for Scratch, as well

Research Desktop

- You can navigate to your Slate storage
  - /N/slate/*username*
- In Caja, you can bookmark this path as well. See image right
- Again, using Bash in the terminal, the ls command shows the same thing



Research Desktop

- The RED interface will not log out automatically if you close the browser tab or desktop client
    - the session will effectively lock the screen
    - this is a good thing, as you can walk away from a session and come back later
    - you will have to reauthenticate when you reconnect
- RED is like having a powerful workstation wherever you go!

Accessing HPC with SSH

- Accessing Research Desktop
- Accessing HPC with SSH
- Finding and Loading Software
- Working with Batch Jobs
- Managing a SLURM Job

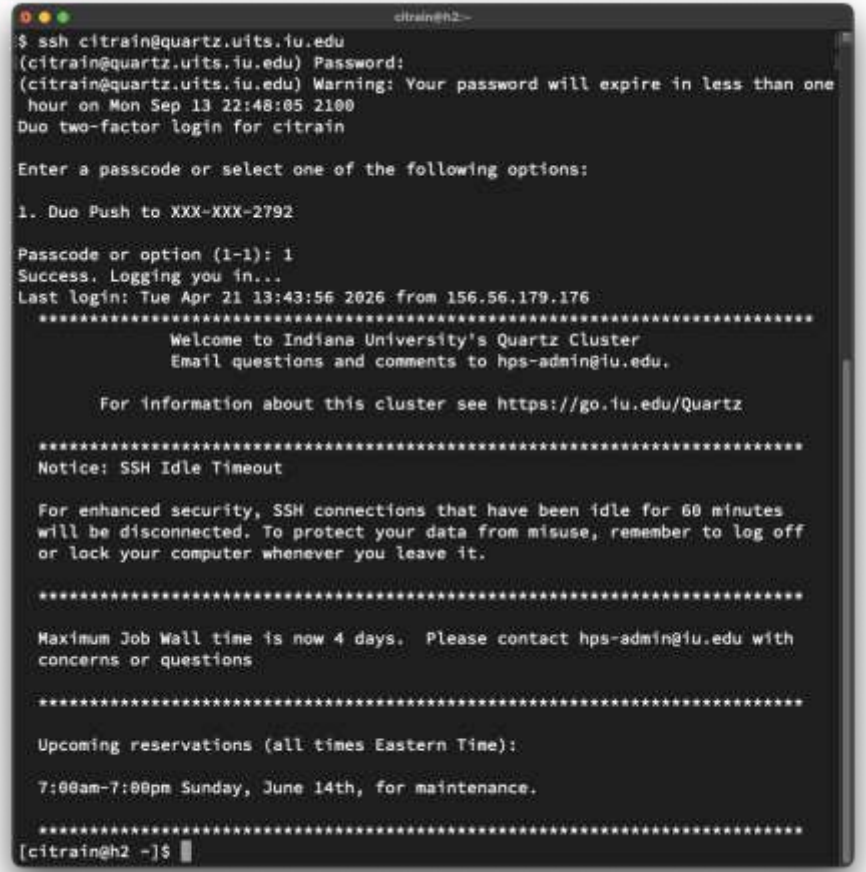
Access HPC with Secure Shell

- In broader HPC environment, accessing via the shell command line in a terminal window is most convenient
- We use Secure Shell (SSH) to do this
  - OpenSSH client is built into Windows 11, MacOS and Linux
- Here's an example to log into Quartz (assuming your username might be different than your laptop/desktop/etc):

```
# ssh username@quartz.uits.iu.edu
```

Using Secure Shell

- Here we have logged in with a username and password
- Then prompted for Duo push
- Then "Message of the Day" and Bash prompt
- See KB for info on setting up keypair authentication and bypassing Duo
  - Search article KB0023919



Finding and Loading Software

- Accessing Research Desktop
- Accessing HPC with SSH
- Finding and Loading Software
- Working with Batch Jobs
- Managing a SLURM Job

Finding and Loading Software

- On the HPC systems, you will have the basic Linux commands available
- A lot of additional software is installed as *modules* in an environment called *Lmod* (a superset of GNU Module)
  - Lmod was written at Texas Advanced Computing Center
- This allows you to configure your environment and use compatible versions of software packages

module is Your Friend

- The command you need to know is *module*
  - module has a number of subcommands, which are necessary
  - **`module avail`** — Lists available packages on the system
  - **`module list`** — Lists packages currently loaded in shell
  - **`module load`** — Loads a package
  - **`module remove`** — Removes a loaded package
  - **`module purge`** — Removes all loaded packages from shell

module example

```
[citrain@h2 ~]$ module list
Currently Loaded Modules:
  1) gnu/9.3.0   2) quota/1.9   3) xalt/3.2.0   4) mvapich/2.3.5   5) StdEnv

[citrain@h2 ~]$ module load python
[citrain@h2 ~]$ module list
Currently Loaded Modules:
  1) gnu/9.3.0    4) mvapich/2.3.5   7) gdbm/1.23
  2) quota/1.9    5) StdEnv          8) openblas/0.3.13
  3) xalt/3.2.0   6) sqlite/3.35.5   9) python/3.12.4

[citrain@h2 ~]$ module purge
[citrain@h2 ~]$ module list
No modules loaded
```

module example (continued)

```
[citrain@h2 ~]$ module reset
Running "module reset". Resetting modules to system default. The following $MODULEPATH directories
have been removed: None
[citrain@h2 ~]$ module list
Currently Loaded Modules:
  1) gnu/9.3.0   2) quota/1.9   3) xalt/3.2.0   4) mvapich/2.3.5   5) StdEnv

[citrain@h2 ~]$ module swap gnu intel
Due to MODULEPATH changes, the following have been reloaded:
  1) mvapich/2.3.5
[citrain@h2 ~]$ module list
Currently Loaded Modules:
  1) quota/1.9   2) xalt/3.2.0   3) StdEnv   4) intel/22.3   5) mvapich/2.3.5
[citrain@h2 ~]$ which icx
/N/soft/rhel8/intel/22.3/compiler/2022.2.0/linux/bin/icx
```

Loading default modules

- If you would like to have modules loaded, swapped or removed automatically when you log in or open a new shell, add the following near the top of your .bashrc file

```
# Source modules
if [ -f ~/.modules ]; then
        . ~/.modules
fi
```

- The lines in .modules should be commands as you would type them on the command line

Working with Batch Jobs

- Accessing Research Desktop
- Accessing HPC with SSH
- Finding and Loading Software
- Working with Batch Jobs
- Managing a SLURM Job

Working with Batch Jobs

- SLURM workload manager is the heart of the HPC batch queue
  - It supports job scheduling, submission, execution, cleanup, accounting, logging
  - Your SLURM commands are communicating with it, not making it run; it is running in the background, all the time
  - You can submit to run immediately with *srun*
    - Your command will wait until SLURM finds the resources for you

Working with Batch Jobs

- You can also submit a job script to the batch queue with *sbatch*
  - Your submission will go into the queue, if there's no error
  - When resources are available for your job, it will run and you will be notified when it completes
  - Standard input, output and error are redirected from/to files in this case
    - Your SLURM script is stdin.
    - You tell SLURM what files to use for stdout and stderr.

SLURM jobs with srun

- Jobs with srun "block," or wait, until they run
  - For certain tasks on a debug queue/partition, this works well

- For example, let's run the *hostname* command in an 8-way parallel job on 8 cores of one server node
  - We expect the hostname to be the same

  - We submit the job with *srun* ($ is the command prompt here):

```
$ srun -A staff -p debug --nodes=1 --cpus-per-task=1 --ntasks=8 hostname
```

SLURM jobs with srun

```
$ srun -A staff -p debug --nodes=1 --cpus-per-task=1 --ntasks=8 hostname
```

- Let's pick apart the command
  - We ask to run with the "staff" project account
    - You will use your RT Project here (might look like "c01845")
  - We have requested the "debug" (cpu-only implied) partition
    - In SLURM lingo, a queue is a "partition"
  - We asked to run 8 tasks on 8 separate cpu cores (in Linux a core is considered a "cpu") on the same (one) node

SLURM jobs with srun

```
$ srun -A staff -p debug --nodes=1 --cpus-per-task=1 --ntasks=8 hostname
```

- SLURM will wait until a node in the debug partition has 8 free cores available to run this job for up to the default amount of wall time
  - wall time means actual clock time (not multiplied by # cores)
  - the default limit on the debug queue is 1 hour
  - you can verify the defaults for any partition with the command

    ```
    $ scontrol show partition partition-name
    ```

SLURM partition defaults

For example:

```
[citrain@h2 ~]$ scontrol show partition debug
PartitionName=debug
   AllowGroups=ALL AllowAccounts=ALL AllowQos=ALL
   AllocNodes=ALL Default=NO QoS=debug
   DefaultTime=01:00:00 DisableRootJobs=NO ExclusiveUser=NO
ExclusiveTopo=NO GraceTime=0 Hidden=NO
   MaxNodes=UNLIMITED MaxTime=01:00:00 MinNodes=0 LLN=NO
MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
   Nodes=c[1-2]
   ...
```

MaxTime is the most you can request. Full syntax is DD-HH:MM:SS

SLURM jobs with srun

- So, what's the output of our *srun* command?

```
$ srun -A staff -p debug --nodes=1 --cpus-per-task=1 --ntasks=8 hostname
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
```

SLURM batch scripts

- The SLURM *sbatch* command will read a "job script" and put the task on the queue, releasing control of your command line
- A SLURM script is effectively a shell script with comments that begin as #SBATCH, rather than just #
- The *sbatch* command interprets the remaining content of those lines as directives
    - They map directly to command line arguments you can use with *sbatch or srun*

SLURM batch scripts

- A batch script for the srun example we just looked at would be

```
#!/bin/bash

#SBATCH --job-name=hostname_test   # Name of the job
#SBATCH --account=staff            # Charge to the staff account (-A)
#SBATCH --partition=debug          # Submit to the debug partition (-p)
#SBATCH --nodes=1                  # Request 1 node
#SBATCH --ntasks=8                 # Run 8 tasks total
#SBATCH --cpus-per-task=1          # Assign 1 CPU core per task
#SBATCH --time=00:10:00            # Set a walltime limit (10 minutes)
#SBATCH --output=hostname_%j.log   # Standard output and error log (%j expands to
JobID)

# Execute the command
# Using srun here ensures Slurm launches 8 instances across the allocated resources
srun hostname
```

SLURM batch scripts

- With no mention of a standard error redirect, the output file will get both stdout and stderr, combined

```
#SBATCH --output=hostname_%j.log   # Standard output and error log (%j expands to JobID)
```

- If you want them separated (recommended), try this

```
#SBATCH --output=hostname_%j.out   # Standard output log
#SBATCH --error=hostname_%j.err    # Standard error log
```

- Be aware that the "%j" in the name will be replaced by the SLURM job id assigned to your submission

SLURM batch scripts

- We submit the job with

```
$ sbatch hostname_test.slurm
Submitted batch job 8808306
```

- and the results look like

```
$ cat hostname_8808306.out
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu
c1.quartz.uits.iu.edu

$ cat hostname_8808306.err
no output
```

UNFINISHED SLIDE (DELETE THIS MARKER WHEN FINISHED)

RED App for SLURM Script

- Placeholder for Robert's app

Managing a SLURM Job

- Accessing Research Desktop
- Accessing HPC with SSH
- Finding and Loading Software
- Working with Batch Jobs
- Managing a SLURM Job

Managing a SLURM job

- When a SLURM job is actively in the partition queue, you can
  - `squeue --me` — Find your own jobs
  - `squeue -p partition` — List jobs in a partition
  - `scancel jobid` — Cancel a job with job id
  - `scontrol hold jobid` — Hold job id from starting
  - `scontrol release jobid` — Release job id to start normally
  - `scontrol show jobid` — Show the state of a job id
  - `scontrol suspend jobid` — Suspend a running job id
  - `scontrol resume jobid` — Resume a suspended job id

Looking closer at partitions

- *sinfo* and *sparta*

$ sparta

Partition	Nodes	DefaultWallTime	MaxWallTime	MaxRunning	MaxQueued	RunningJobs	QueuedJobs	Unscheduled
debug	2	01:00:00	01:00:00	1	2			
general	88	01:00:00	4-00:00:00	1000	973	72	25	
interactive	2	00:30:00	08:00:00	2	16	1		
gpu-debug	2	00:30:00	01:00:00	1	2			
gpu	22	01:00:00	2-00:00:00	36	376	313		
gpu-interactive	2	00:30:00	04:00:00	1				
hopper	12	01:00:00	2-00:00:00	1000	18	43	18	

Reflection

- You now know how to:
  - Access Research Desktop via browser or ThinLinc
  - SSH into Quartz from any terminal
  - Load and manage software modules with `module`
  - Submit interactive jobs with `srun`
  - Submit batch scripts with `sbatch`
  - Monitor and manage running jobs with `squeue` and `scontrol`
- Next module covers HPC storage systems
- Be sure to complete the reflection for this lesson